

THE MAGIC MIRROR: INTRODUCTION TO RECURSION

1. The Magical Mirror: What is Recursion?

Story Time: Vikram's Magical Mirror Palace

Vikram found a magical mirror in an ancient palace that showed smaller mirrors inside, going on and on! Imagine standing between two mirrors and seeing yourself get smaller and smaller into infinity. Russian nesting dolls (Matryoshka dolls) work the exact same way: you open one doll, and inside is a smaller version of the same doll!

In computer programming, **Recursion** is a technique where a function calls **itself** to solve a problem by breaking it down into smaller, manageable pieces that look just like the original problem.

- **Self-Reference:** A recursive function solves a complex problem by invoking itself with smaller inputs.
- **Similar Sub-problems:** Each sub-problem has the exact same structure as the main problem.
- **Reduction:** Step by step, the problem reduces until it reaches the simplest possible answer.

2. The Two Golden Rules of Recursion

Just like traffic lights control cars on a road, every recursive function must follow two golden rules to work safely and avoid crashing:

Rule	Signal	Description
1. Base Case	RED LIGHT (STOP)	The condition that stops the function from calling itself. It provides the direct answer for the simplest case.
2. Recursive Case	GREEN LIGHT (GO)	The part of the function where it calls itself with a smaller input, moving closer toward the base case.

3. The Call Stack: Steel Tiffin Boxes

Real-World Analogy: Stacked Tiffin Boxes

Imagine stacking colourful steel tiffin boxes on top of each other:

- **Pushing to Stack:** Function Call 1 goes on the bottom, then Call 2 sits on top of it, then Call 3...
- **Popping from Stack:** The top tiffin box (the last function call) must finish first before you can open the ones beneath it!
- Python remembers each step on the *Call Stack* until the base case is reached. Once the base case finishes, the stack unwinds!

4. Code Example: The Laddu Distribution Challenge

Suppose Vikram needs to distribute n laddus to children one by one using a recursive Python function:

```
def give_laddus(n):  
    if n == 0: # Base Case: Stop when no laddus remain!  
        return  
  
    print("Laddu!") # Action: Give one laddu  
    give_laddus(n - 1) # Recursive Case: Pass remaining (n - 1) laddus
```

What Happens If We Forget the Base Case?

If a recursive function lacks a base case (or never reaches it), it will call itself infinitely until Python runs out of stack memory!

Result: RecursionError: maximum recursion depth exceeded.

5. Lesson Summary & Recap

- **Recursion:** A function calling itself to solve smaller sub-problems (like mirrors inside mirrors).
- **Base Case vs. Recursive Case:** The base case stops the loop; the recursive case drives forward toward the base case.
- **The Call Stack:** Manages function execution in a Last-In, First-Out manner (like stacked tiffin boxes).

STUDENT PRACTICE QUESTIONS

Test Your Knowledge on Python Recursion

Answer the following questions based on the concepts covered in this lesson. (Do not include answers on this sheet).

Question 1 (Multiple Choice)

What is recursion in computer programming?

- A) A loop that runs a fixed number of times using a range() function
- B) A function that calls itself to solve smaller instances of the same problem
- C) A method to convert Python code into executable files
- D) A data structure that stores items in key-value pairs

Question 2 (Multiple Choice)

In the Laddu Distribution function, which line represents the **Base Case**?

- A) `def give_laddus(n):`
- B) `if n == 0: return`
- C) `print("Laddu!")`
- D) `give_laddus(n - 1)`

Question 3 (Multiple Choice)

What error does Python raise if a recursive function calls itself indefinitely without hitting a base case?

- A) `TypeError`
- B) `IndentationError`
- C) `RecursionError: maximum recursion depth exceeded`
- D) `ZeroDivisionError`

Question 4 (Analogy & Short Answer)

Explain how a stack of steel tiffin boxes illustrates the working of the **Call Stack** when executing recursive functions.

Question 5 (Code Completion / Challenge)

Complete the missing recursive call in the following starter code to calculate the sum of numbers from 1 to N:

```
def sum_to_n(n):  
    if n == 1: # Base case  
        return 1  
    return n + sum_to_n( _____ ) # Fill in the blank!
```

Question 6 (Short Answer)

Why is it essential for every recursive function to have BOTH a base case and a recursive case? What happens if either one is missing?